

# Backstop: In-Process Transport-Layer Guardrails for LLM Applications

Ravani Roshan and the Backstop Contributors

Backstop v0.5.0 — September 15, 2026 — MIT License — [github.com/RavaniRoshan/backstop](https://github.com/RavaniRoshan/backstop)

**Abstract.** Large-language-model (LLM) applications fail in an expensive way: a single runaway loop or a parallel multi-agent fan-out can multiply token spend before any operator notices. Today’s protective options are either nothing at all, or a proxy gateway that adds a network hop, a deployed service, and an operational dependency to every call. We present *Backstop*, a guardrail layer that lives *inside the application process* by replacing the `httpx` transport of the official OpenAI and Anthropic SDKs — no proxy, no monkey-patching, one line of code. Backstop enforces reserve-then-reconcile token budgets, priority admission with starvation prevention, additive-increase/multiplicative-decrease (AIMD) concurrency control, retry with backoff, and a circuit breaker, in that order, before any request leaves the process. We evaluate with a deterministic seeded harness plus live-provider proofs: control-path overhead is **0.12 ms p50** (0.20 ms p99) against a mock provider; a live 500-token budget allowed exactly 2 calls and blocked 18; two agents with separate budgets were isolated from each other’s exhaustion; and a bundled multi-agent tool (Wedge) shows per-agent isolation combined with a convergence measurement (similarity 0.98, verdict PARTIAL). A zero-dependency dashboard renders the same telemetry, and the sink behind it costs one null-check per event when unused. Backstop is open source (Python 3.10+, TypeScript SDK), and every number in this paper is reproducible from the repository.

**Keywords:** LLM cost control — budget enforcement — congestion control — circuit breaker — in-process middleware — multi-agent systems

## 1. Introduction

Running one LLM-backed agent is already unpredictable in cost and latency. Running several in parallel — the shape multi-agent architectures require — multiplies the exposure. Production teams typically discover spend controls *after* the first incident. The two known mitigations are (a) hope the agent stops, or (b) deploy a proxy gateway and route all traffic through it. The first is not a control; the second buys safety with a new distributed-systems problem: a network hop on every call, a service to deploy and scale, and a new point of failure on the hot path.

Backstop takes a third path: **SDK-native guardrails inside your process**. The core observation is that the official OpenAI and Anthropic Python SDKs both accept a custom `http_client`, and every request flows through an `httpx` transport chain. Backstop supplies its own transport, so every request is intercepted in-process before dispatch — using the SDK’s own extension point, not monkey-patching.

This paper makes the following contributions:

- **Design.** A pipeline of transport-layer guardrails — budget reservation, priority admission, AIMD concurrency, circuit breaking, retry — arranged so prevention happens *before* dispatch and reconciliation *after* response (Section 3).
- **Implementation.** A drop-in `Backstop.wrap(client)` for OpenAI and Anthropic (sync and async): streaming support, tenant buckets, exact and semantic caching, in-process fallback chains, an opt-in cross-replica shared budget, Prometheus/OpenTelemetry export, and a built-in dependency-free dashboard (Section 4).
- **Evaluation.** A deterministic seeded harness whose request/block counts are exact and reproducible, live-provider proofs of budget exhaustion and isolation, measured sub-millisecond control-path overhead, and a multi-agent diff study (Wedge) tying per-agent budget isolation to a measurable convergence verdict (Section 5).
- **Honesty as a feature.** Every figure carries its methodology, and the product states what it does *not* measure (Section 7).

## 2. Motivation and Problem Model

Consider an application issuing  $N$  provider calls through a loop with no guardrails. Worst-case cost is unbounded: a misparsed response, a retry loop, or an agent that keeps calling tools until context overflows. With three parallel agents the exposure triples, and no mainstream SDK offers a first-class per-agent cap.

A proxy gateway solves enforcement but changes the deployment shape: every call gains a network round-trip (typically 10–100 ms), one more service must be operated, and the application must trust an external component on its hot path. Backstop’s hypothesis is that *the transport layer is the right interception point* — low enough to see every byte crossing to the provider, high enough to remain SDK-agnostic and to know per-request semantics (model, tokens, priority, tenant).

Two requirements follow. First, enforcement must be *provable*: a budget that is “mostly respected” is not a budget, so reservation happens before dispatch and reconciliation after the response, with the reservation refunded on failure. Second, the guardrail must be *observable without infrastructure*: an operator with no Grafana must still be able to answer “what is Backstop doing right now?” from the same process.

### 3. Design

#### 3.1. Guardrail pipeline

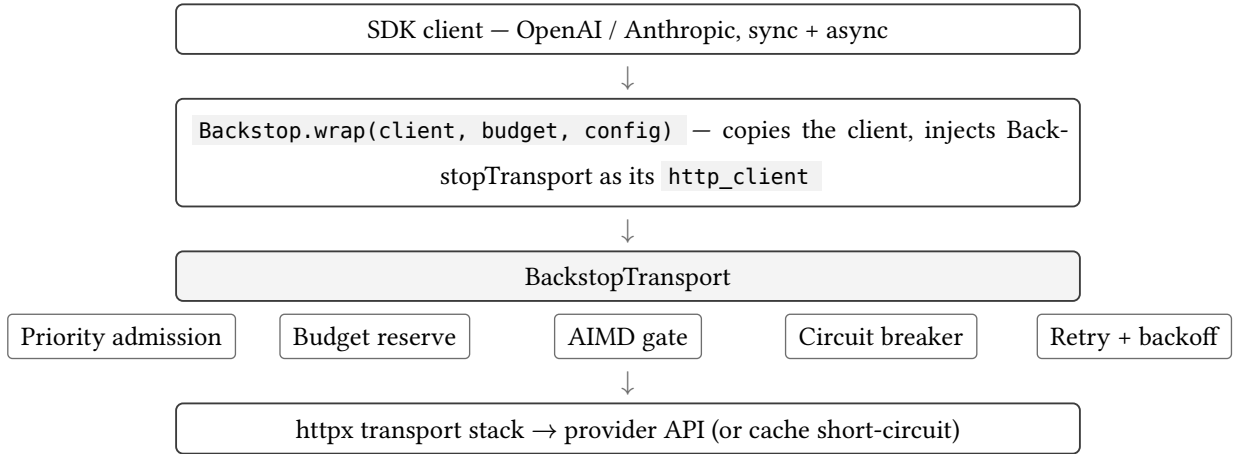


Figure 1: The Backstop pipeline. Each stage sits on the SDK’s native httpx transport path; requests that fail a stage never leave the process.

Figure 1 shows the pipeline. Cheap rejections (circuit open, priority starvation) come before budget reservation so a blocked call never consumes budget bookkeeping, and budget checks do not queue behind each other. Every stage is instrumented through one choke point (`Metrics.call`), which feeds the telemetry sink, Prometheus, and OpenTelemetry alike.

#### 3.2. Reserve-then-reconcile budgets

For each call Backstop estimates the token cost (prompt heuristic plus `max_tokens`), *reserves* that amount against the session budget, dispatches, then *reconciles* against actual usage from the provider: the difference is refunded on over-estimate and enforced on under-estimate. When the reservation cannot be granted, the request is blocked locally and its estimated cost is counted as *prevented* spend — the provider call never happens. Streaming responses are reconciled incrementally as chunks arrive, so a stream that outgrows its reservation is cut off mid-flight rather than after completion.

#### 3.3. Priority admission and AIMD concurrency

Requests carry a priority — `critical`, `default`, or `background` — and the gate admits them with starvation prevention, so a flood of background work cannot wedge a critical call. Concurrency is governed by an additive-increase/multiplicative-decrease controller in the classic sense [1]:

$$c_{n+1} = \begin{cases} \min(c_{\max}, c_n + \alpha) & \text{on success} \\ \beta \cdot c_n & \text{on loss / pressure} \end{cases}$$

with `aimd_adjustment_interval` pacing updates. The controller treats provider pressure — 429s, latency inflation, circuit events — as congestion signal, exactly as TCP treats loss. A bounded admission queue reports depth and wait time, observed per priority.

### 3.4. Circuit breaker and in-process fallback

A failure-rate threshold with a cooldown timer drives the breaker through CLOSED / HALF\_OPEN / OPEN [2]. While open, requests are blocked in-process — counted as prevented — rather than forwarded into a failing provider. On sustained failure Backstop walks an ordered `fallback_chain` of backup models or deployments *inside the process*, with an optional priority-specific chain for critical traffic; this recovers the availability a gateway would otherwise provide, without the gateway.

### 3.5. Tenants and multi-session isolation

Each `Backstop.wrap` session owns an independent budget, AIMD limit, breaker, and ledger; a request may additionally carry a tenant bucket via `with_budget(...)`, giving request-scoped sub-budgets under the session cap. Sessions are tracked by weak reference: a session is visible exactly as long as its owner holds it, which is what makes per-agent isolation *observable* (Sections 4.3 and 5.4).

## 4. Implementation

### 4.1. The one-line wrap

```
from openai import OpenAI
from backstop import Backstop

client = Backstop.wrap(OpenAI(), budget=50_000)
# use the client exactly as before – guardrails intercept at the transport layer
```

The same call works for Anthropic, and for async clients via `wrap_async`. `wrap` copies the client (preserving headers, default query, retries, base URL), injects `BackstopTransport`, and returns the same type it was given — IDE completion, streaming, and SDK updates keep working. A TypeScript SDK (`@ravanish/backstop`) mirrors the API for Node.js agents.

### 4.2. Caching: exact and semantic

An optional in-memory cache serves exact duplicate prompts with zero cost; the opt-in **semantic cache** compares prompt embeddings (pluggable embedder, cosine similarity  $\geq$  `cache_similarity_threshold`) and short-circuits near-duplicates without a provider call. At the 30–50% near-duplicate rates typical of RAG workloads, the documentation claims 50–80% token savings on cached traffic [4].

### 4.3. Shared budget across replicas

For multi-process deployments, an opt-in Redis mode enforces *one* token budget across replicas with atomic Lua scripts — no Postgres, no gateway, no change to the call path. The default remains purely in-process with zero infrastructure.

### 4.4. Telemetry and the built-in dashboard

All counters, gauges, and latency observations flow through a single choke point, so Backstop can feed three consumers without changing call sites: a dependency-free telemetry sink (the dashboard's source), an optional Prometheus registry, and an optional OpenTelemetry meter. The sink is `None` unless a dashboard is running, making the unused cost one `is None` comparison per event. The dashboard itself is stdlib-only WSGI: budget burn and projected exhaustion, prevention mix, traffic and provider efficiency, latency p95, AIMD state, per-session isolation rows, tenant ledger, and the enforcement event stream — loopback-only unless a bearer token is set, `GET`-only, strict CSP, no external origins.

## 5. Evaluation

### 5.1. Deterministic harness results

The benchmark harness (`backstop benchmark`, seed `0xC0FFEE`) uses a local mock transport: no network, exact counts, reproducible with one command. Overhead is measured *separately* from provider latency (Table 1).

| Metric      | Direct  | Backstop | Overhead       |
|-------------|---------|----------|----------------|
| p50 latency | 0.13 ms | 0.25 ms  | <b>0.12 ms</b> |
| p95 latency | 0.28 ms | 0.43 ms  | <b>0.15 ms</b> |
| p99 latency | 0.36 ms | 0.56 ms  | <b>0.20 ms</b> |

Table 1: Control-path overhead on a local mock transport (1000 requests, seeded). Latency is measured separately from provider latency.

| Scenario     | Requests | Prov. calls | Successes | Prov. errors | Budget-blk | Circuit-blk |
|--------------|----------|-------------|-----------|--------------|------------|-------------|
| burst        | 50       | 50          | 50        | 0            | 0          | 0           |
| steady-state | 30       | 30          | 30        | 0            | 0          | 0           |
| error-storm  | 50       | 12          | 8         | 0            | 0          | <b>42</b>   |
| budget-hit   | 80       | 16          | 16        | 0            | <b>64</b>  | 0           |

Table 2: Seeded harness scenarios. All counts are exact and reproducible via `backstop benchmark`. “Blocked” requests never reached the provider.

In *budget-hit*, 64 of 80 requests were blocked before dispatch — the budget cap held exactly. In *error-storm*, the breaker and AIMD controller blocked 42 of 50 requests, all counted as prevented rather than forwarded into a failing provider.

## 5.2. Live-provider proofs

Beyond the harness, proofs were run against a live OpenAI-compatible provider (OpenCode Zen, DeepSeek V4 Flash) [4]:

| Agent        | Budget | Allowed | Blocked | Spent | Remaining |
|--------------|--------|---------|---------|-------|-----------|
| A (tight)    | 300    | 1       | 9       | 300   | 0         |
| B (generous) | 5000   | 4       | 6       | 5000  | 0         |

Table 3: Live multi-agent isolation proof: each `wrap()` session enforced its own budget; Agent A’s exhaustion did not affect Agent B’s cap.

A second live proof set a 500-token budget against 2560 tokens of attempted work: exactly 2 calls were allowed (448 tokens spent) and 18 blocked — prevention *before* the first overspent call, not detection after. On a 13.2 s reasoning-model call, Backstop added 1.1 s (9%) — dominated by the model’s own reasoning and retry behavior; the mock-measured control path is sub-millisecond (Table 1). A proxy gateway adds a 10–100 ms network round-trip to *every* call instead.

## 5.3. Wedge: per-agent isolation and convergence

Wedge, bundled with Backstop, runs  $n$  isolated coding agents on the same task — each with its own `wrap` session (budget 20000), working directory, and no shared conversation history — then diffs their patches and scores convergence (CONVERGED / PARTIAL / DIVERGED). In the committed run, 3 Anthropic runners produced patches with similarity 0.98 (verdict PARTIAL), tests passing 3/3, and per-runner budget usage recorded in the report. The report doubles as evidence that isolation holds *under convergence pressure*: each runner’s spend stopped exactly at its own cap.

## 6. Discussion and Limitations

Backstop’s operating range is defined by the transport layer. It does *not* read application-level signals: it cannot see “disagreement” or “confidence” across agents (that is Wedge’s job, post hoc), and it deliberately does not store or query metrics — it exports Prometheus and OpenTelemetry instead. Multi-provider routing and centralized key vaulting — the strengths of proxy gateways — are non-goals. If an organization already operates a gateway, Backstop composes with it: enforcement at the client adds a defense the gateway cannot provide per agent.

Reserve-then-reconcile is conservative by construction: the estimate must cover `max_tokens`, so short responses refund the difference but a request that would exceed the remaining budget is blocked even when the real response

would have been smaller. Deterministic harness numbers use a mock provider and measure the control path only; live-provider runs (13.2 s calls, 1.1 s added) are dominated by model variance and should be read as within-variance, not additive. The semantic cache requires a user-supplied embedder and inherits its quality; the shared-budget mode requires a reachable Redis and its atomicity is per-script, not per-transaction.

## 7. Related Work

**Proxy gateways.** LiteLLM and BricksLLM enforce budgets, keys, and routing as deployed proxies [3]; every call leaves the process and gains a hop. Backstop’s comparative matrix positions itself where gateways are partial (per-agent transport-level budget reserve, AIMD, in-process fallback) or absent entirely (reproducible seeded benchmarks, provable per-agent isolation). **Congestion control.** AIMD and the AIMD-inspired concurrency control have decades of history from TCP [1]; Backstop applies the same control law to provider pressure. **Resilience patterns.** The circuit breaker follows the classic state-machine formulation [2]; the contribution here is placing it, with budgets, on the SDK transport path of *existing* clients rather than behind a new service boundary.

## 8. Conclusion

Backstop shows that the essential guardrails of an LLM gateway — budget caps, admission control, congestion response, failure isolation — can be moved *into the process*, onto an SDK extension point that was already there, at sub-millisecond cost and one-line adoption. Its evaluation is unusual for a developer tool: seeded deterministic counts, live-provider proofs, and a multi-agent study that measures both isolation and convergence. The result is a guardrail that proves its own claims — prevention you can reproduce, not promises you must trust.

## 9. References

- [1] V. Jacobson, “Congestion Avoidance and Control,” *ACM SIGCOMM*, 1988.
- [2] M. Fowler, “Circuit Breaker,” *Patterns of Enterprise Application Architecture* / *martinfowler.com*, 2014.
- [3] BerriAI, “LiteLLM — Open Source AI Gateway,” [github.com/BerriAI/litellm](https://github.com/BerriAI/litellm); BricksCloud, “BricksLLM — Enterprise-grade API Gateway,” [github.com/bricks-cloud/BricksLLM](https://github.com/bricks-cloud/BricksLLM). Accessed 2026-07-20.
- [4] Backstop repository: README, docs/benchmark-results-2026-08-10.md, docs/proof-evidence-2026-08-10.md, docs/competitive-benchmark-2026-07-20.md, docs/dashboard.md. [github.com/RavaniRoshan/backstop](https://github.com/RavaniRoshan/backstop), v0.5.0.